



OpenSpec: Spec-Driven Development nell'era degli Agenti AI

Un nuovo paradigma per la collaborazione tra Umani e Intelligenza Artificiale

Matteo Baccan

DevFest Modena 2026 - 3/4 Ottobre 2026, Modena

Track: AI & Machine Intelligence



Google
Developer
Groups

Di cosa parleremo

01 **Dal Vibe Coding a OpenSpec**

Perché i prompt improvvisati non scalano e serve una struttura

03 **Il Linguaggio dell'Intento**

EARS + BDD: specifiche che l'agente non può fraintendere

05 **Dal Team al Legacy**

Adozione incrementale sul brownfield e anti-pattern da evitare

02 **La Meccanica di OpenSpec**

Directory, artefatti e Delta Specs: l'anatomia del framework

04 **Il Ciclo di Esecuzione**

Propose → Apply → Archive: la macchina a stati delle modifiche

06 **Perché Tutto Questo Conta**

Determinismo, intento persistente e il ruolo dell'Agentic Engineer



Google
Developer
Groups

01

Dal Vibe Coding a OpenSpec

Perché i prompt improvvisati non scalano e serve una struttura



Google
Developer
Groups

Il Problema: Agenti AI senza struttura

Gli agenti AI sono collaboratori attivi, ma senza struttura generano **vibe coding**: prompt informali, contesto volatile, drift dei requisiti.

- I requisiti vivono solo nella chat: nessuna documentazione persistente
- Allucinazioni architetturali e scelte inconsistenti
- Debito tecnico invisibile e difficilmente tracciabile



Google
Developer
Groups

La Soluzione: Spec-Driven Development

Lo **Spec-Driven Development** inverte il paradigma: **la struttura prima del codice**.

	Vibe Coding	Spec-Driven Development
Flusso	Prompt → Codice	Idea → Specifica → Architettura → Task → Codice
Artefatti	Nessuno (solo chat)	<code>proposal.md</code> , <code>design.md</code> , <code>tasks.md</code> , Delta Specs
Verificabilità	Impossibile	Ogni fase produce output controllabile



Google
Developer
Groups

OpenSpec: i tre pilastri

Pilastro 1: Brownfield-First ($1 \rightarrow n$)

Ottimizzato per l'evoluzione di codebase esistenti, non solo per prototipi greenfield ($0 \rightarrow 1$).

Permette di modificare logiche complesse in sicurezza, senza riscrivere tutto da zero.

Pilastro 2: Architettura Leggera

Markdown + Git come base operativa: niente infrastruttura pesante, niente database complessi.

Il metodo resta trasparente, versionabile e vicino al flusso reale del team.

Pilastro 3: Agnosticismo Totale

Zero lock-in verso IDE, modelli o vendor: il valore vive nelle specifiche, non nella piattaforma.

Se cambi agente o ambiente, il processo resta intatto perché l'intento rimane nel repository.

La specifica è la fonte della verità, non il codice: i documenti sono istruzioni eseguibili e vincolanti per l'agente, non suggerimenti.

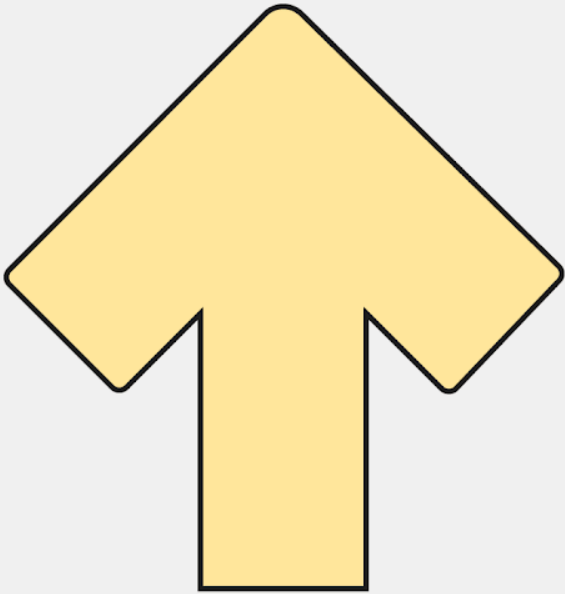


Google
Developer
Groups

02

La Meccanica di OpenSpec

Directory, artefatti e Delta Specs:
l'anatomia del framework



Google
Developer
Groups

Un'unica directory: la memoria dell'agente

Tutto risiede in `openspec/`, la memoria a lungo termine dell'agente AI, con due anime separate:

Cartella	Ruolo	Contenuto
<code>specs/</code> (l'Essere)	La Source of Truth	Comportamento attuale del sistema, per domini logici (<code>auth/</code> , <code>payments/</code> , <code>ui/</code>)
<code>changes/</code> (il Divenire)	Il workspace di modifica	Una cartella isolata per ogni feature o bug fix, senza conflitti finché non si archivia

Questa distinzione garantisce transizioni sicure e modifiche parallele.



Google
Developer
Groups

Governance attiva: come `config.yaml` guida ogni richiesta

<code>config.yaml</code>	Funzione	Effetto sull'agente
<code>schema</code>	Definisce workflow e struttura degli artefatti attesi	L'agente non improvvisa formati: produce output conformi
<code>context</code>	Fornisce architettura globale, stack, standard tecnici	Ogni piano parte dagli stessi vincoli di progetto
<code>rules</code>	Impone guardrail granulari per fase e artefatto	Riduce deviazioni, allucinazioni e scelte fuori policy

Risultato operativo: il file non resta passivo nel repository, viene iniettato nel prompt di pianificazione e rende la conformità tecnica sistematica.



Google
Developer
Groups

Anatomia end-to-end di una proposta

Fase	File	Domanda chiave	Ruolo operativo
1	proposal.md	Perché / Cosa stiamo cambiando?	Definisce intento strategico e ambito lavori (business case iniziale).
2	design.md	Come lo realizziamo?	Fissa decisioni tecniche: architettura, flussi dati, librerie e vincoli.
3	tasks.md	Come eseguiamo in modo verificabile?	Scompone in task atomici e sequenziali, eseguibili dall'agente senza ambiguità.
4	Delta Specs	Cosa cambia nella Source of Truth?	Applica patch ai requisiti con sezioni ADDED , MODIFIED , REMOVED .

Flusso completo: proposal.md → design.md → tasks.md → Delta → consolidamento → specs/



Google
Developer
Groups

L'intento in pratica: `proposal.md`

Cattura l'intento strategico: "perché stiamo facendo questa modifica?". È il punto di allineamento iniziale tra umano e macchina.

Why

Gli utenti abbandonano la registrazione: serve il login social.

What Changes

- Aggiunta autenticazione OAuth (Google) al flusso di login
- Nessun impatto sul login email/password esistente



Google
Developer
Groups

Le decisioni in pratica: `design.md`

È l'ancora tecnica: fissa librerie, dati e pattern, così l'agente non "immagina" soluzioni creative ma errate.

Decisioni tecniche

- Libreria: passport con strategy passport-google-oauth20
- Nel DB solo l'id utente Google: nessun token persistito
- Sessioni: riuso del meccanismo esistente (Redis)



Google
Developer
Groups

L'esecuzione in pratica: `tasks.md`

Una checklist di azioni atomiche: l'agente aggiorna le spunte in tempo reale mentre scrive il codice, garantendo totale trasparenza.

1. Backend

- [x] 1.1 Aggiungere endpoint GET /auth/google
- [x] 1.2 Gestire il callback e creare l'utente
- [] 1.3 Test di integrazione del flusso completo



Google
Developer
Groups

Delta Specs: patch per i requisiti

La vera innovazione: invece di riscrivere l'intera specifica, l'agente descrive solo cosa cambia: meno token, più accuratezza.

- **ADDED Requirements**: comportamenti completamente nuovi
- **MODIFIED Requirements**: requisiti riscritti per intero, senza perdere regole pregresse
- **REMOVED Requirements**: funzionalità deprecate, con motivo e migrazione

MODIFIED Requirements

Requirement: Session Duration

The system SHALL scadere la sessione dopo 30 minuti di inattività (prima: 24 ore).



Google
Developer
Groups

03

Il Linguaggio dell'Intento

EARS + BDD: specifiche che l'agente non può fraintendere



Google
Developer
Groups

Il linguaggio dell'intento: EARS + BDD

Asse	EARS (obbligazione)	BDD (esecuzione)
Scopo	Ridurre ambiguità nei requisiti	Rendere verificabili i comportamenti
Forma	Parole chiave normative: SHALL , MUST , SHOULD	Struttura scenario: GIVEN , WHEN , THEN
Domanda a cui risponde	"Cosa è obbligatorio?"	"Come si osserva il risultato?"
Esito pratico	Contratti chiari per l'agente	Test di accettazione derivabili

Regola operativa: prima definiamo il vincolo con EARS, poi ne proviamo l'esecuzione con BDD.



Google
Developer
Groups

Sintassi EARS

OpenSpec adotta l'approccio EARS (Easy Approach to Requirements Syntax).

L'obiettivo è minimizzare l'ambiguità utilizzando parole chiave vincolanti per definire i livelli di obbligazione.

Non nasce nel software. EARS viene da Rolls-Royce: Alistair Mavin e colleghi la ricavarono analizzando le normative di aeronavigabilità del sistema di controllo di un motore a reazione, e la pubblicarono a IEEE RE'09 nel 2009.

Oggi è adottata da Airbus, Bosch, Honeywell, Intel, NASA e Siemens: la stessa sintassi che tiene in volo un motore ora vincola un agente AI.



Google
Developer
Groups

Dallo scenario al test

Le specifiche descrivono il comportamento osservabile, non i passi di implementazione: sono contratti di business, non tutorial di codice.

- **GIVEN:** Il contesto iniziale (es. "Dato un utente non autenticato").
- **WHEN:** L'azione scatenante (es. "Quando visita la dashboard").
- **THEN:** Il risultato atteso (es. "Allora viene reindirizzato al login").

Con scenari così strutturati, l'agente genera automaticamente i test unitari o e2e corrispondenti, chiudendo il ciclo della qualità.

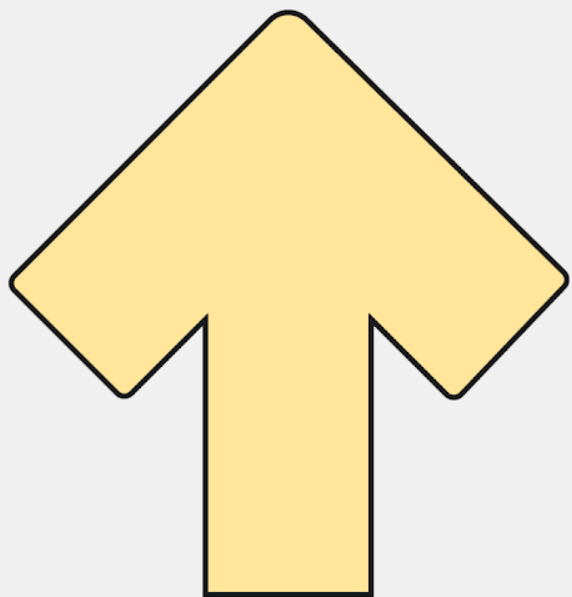


Google
Developer
Groups

04

Il Ciclo di Esecuzione

Propose → Apply → Archive: la macchina a stati delle modifiche



Google
Developer
Groups

La macchina a stati di OpenSpec

Fase	Obiettivo	Output della fase	Gate di passaggio
Propose	Allineare intento e ambito prima del codice	Cartella in <code>changes/</code> con <code>proposal.md</code> , <code>design.md</code> , <code>tasks.md</code> , Delta Specs	Revisione umana dell'intento
Apply	Implementare senza deviazioni dai vincoli	Codice + test aderenti a <code>design.md</code> e <code>tasks.md</code>	Verifica qualità su MUST/SHALL
Archive	Consolidare la modifica nella verità di sistema	Fusione Delta Specs in <code>specs/</code> + archivio storico della change	Stato aggiornato e audit trail persistente

Logica del ciclo: cattura presto il disallineamento (Propose), esegui in modo deterministico (Apply), consolida nella Source of Truth (Archive).



Google
Developer
Groups

La Revisione dell'Intento

Questo è il momento chiave per lo sviluppatore umano.

Modificare un file Markdown errato richiede pochi secondi. Correggere un'architettura software errata dopo che è stata scritta richiede giorni.



Google
Developer
Groups

Focalizzazione: il 100% su un compito alla volta

Con OpenSpec ogni fase riceve l'intera capacità computazionale del modello:

Fase	Focus dell'agente
Propose	100% sull'architettura e le decisioni di design
Apply	100% sulla correttezza e completezza del codice
Archive	100% sulla coerenza documentale e la Source of Truth

Più focus su un compito → Meno errori → Meno cicli di correzione



Google
Developer
Groups

Parallelismo e sincronizzazione

OpenSpec scala su tre assi:

Orizzontale: branch paralleli

Più agenti su feature diverse, ognuno con la propria cartella `changes/`; il ciclo `Propose → Apply → Archive` guida il merge ordinato nella Source of Truth.

Verticale: Project Management

Via MCP dialoga con Linear/Jira: il backlog resta allineato allo stato reale del codice.

Cross-repo: Stores (beta)

La pianificazione vive in un repository dedicato e condiviso; i repo di codice si allineano dopo.

Messaggio chiave: OpenSpec scala in orizzontale, in verticale e tra repository



Google
Developer
Groups

I Ruoli nel Sistema Multi-Agente

Agente Architetto

Consuma `config.yaml` e produce Record di Decisione Architetture (ADR).

Genera la struttura iniziale OpenSpec e pianifica la suddivisione logica del sistema.

Agente Orchestratore

Legge il `tasks.md` prodotto dall'architetto e delega il lavoro ai team specializzati.

Supervisiona l'avanzamento senza scrivere codice direttamente.

Team di Sviluppo AI

Agenti iperspecializzati per dominio: Data Access, Business Logic, Frontend.

Lavorano in parallelo sui propri task atomici, vincolati alle specifiche.

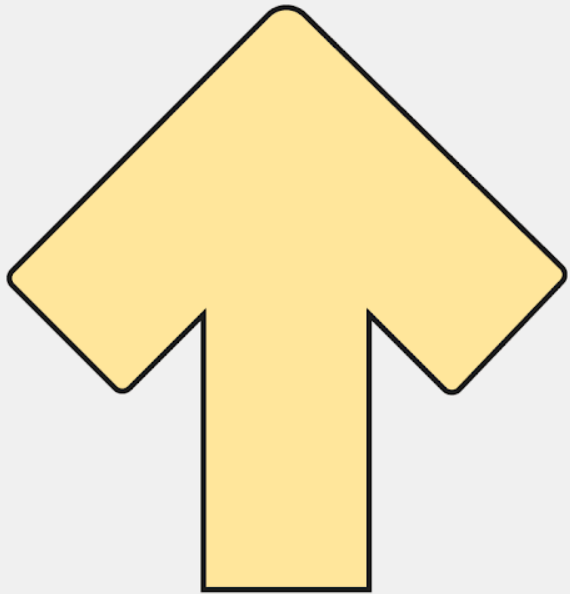


Google
Developer
Groups

05

Dal Team al Legacy

Adozione incrementale sul brownfield
e anti-pattern da evitare



Google
Developer
Groups

Brownfield adozione incrementale

La scala dell'adozione	<code>/opsx:onboard</code> (reverse engineering)
1. Non fermare il mondo Nessun bisogno di produrre specifiche upfront per milioni di righe.	Comando dedicato per accelerare la partenza su moduli complessi o poco documentati.
2. Sviluppo just-in-time Usa OpenSpec sulla prossima feature o sul bug critico, dove stai già intervenendo.	L'agente analizza il codice esistente per estrarre le regole di business già implementate.
3. Accumulo organico La Source of Truth cresce naturalmente ad ogni ciclo <code>Propose -> Apply -> Archive</code> .	Generazione assistita di file <code>spec.md</code> retroattivi, da rifinire e validare con il team.

Per partire: `npm install -g @fission-ai/openspec` e `openspec init`



Google
Developer
Groups

Mitigazione dei rischi: 4 anti-pattern da evitare

Il cimitero delle proposte

Problema: dimenticare `Archive` e lasciare change mai consolidate.

Fix: integrare l'archiviazione nel workflow operativo e nei controlli di qualità.

Il micro-management dell'AI

Problema: scrivere dettagli implementativi nelle specifiche principali.

Fix: tenere il comportamento nelle spec e spostare il "come" in `design.md`.

Burocrazia per modifiche banali

Problema: usare una SDD completa per un cambio minimo e non rischioso.

Fix: applicare progressive rigor e mantenere un approccio leggero.

Ignorare la formattazione Delta

Problema: non usare `ADDED`, `MODIFIED`, `REMOVED` in modo strutturato.

Fix: rispettare i metadati Delta per merge puliti e consolidamento affidabile.



Google
Developer
Groups

06

Perché Tutto Questo Conta

Determinismo, intento persistente e
il ruolo dell'Agentic Engineer



Google
Developer
Groups

Impatto strategico

Determinismo + intento persistente = scalabilità umano-AI

Precisione	Resilienza	Evoluzione del ruolo
Specifiche eseguibili: più accuratezza al primo tentativo, meno rilavorazioni.	La conoscenza architettuale non muore nella chat: resta al team e resiste al turnover.	Il valore umano passa dallo "scrivere tutto" al garantire che il codice sia quello necessario.

Sintesi: quando l'implementazione è deterministica e l'intento rimane persistente, il sistema scala senza perdere qualità.

Il codice è l'output. La specifica è la competenza.



Google
Developer
Groups

Cosa porti a casa da domani

- **Installa:** `npm install -g @fission-ai/openspec`, poi `openspec init` nel tuo progetto esistente
- **Parti piccolo:** documenta solo la prossima feature o il prossimo bug critico, niente Big Bang documentale
- **Scrivi per l'agente:** requisiti EARS (`SHALL` / `SHOULD`) + scenari `GIVEN` / `WHEN` / `THEN` verificabili
- **Separa le fasi:** prima rivedi la proposta (secondi), poi lascia scrivere il codice (l'architettura sbagliata costa giorni)
- **Archivia sempre:** le Delta Specs fuse in `specs/` sono la tua Source of Truth, non la chat



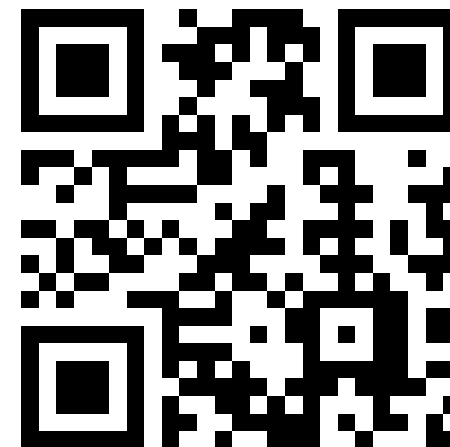
Google
Developer
Groups

Grazie! Ask Me Anything



Matteo Baccan

Rivoluzione Digitale
head of development

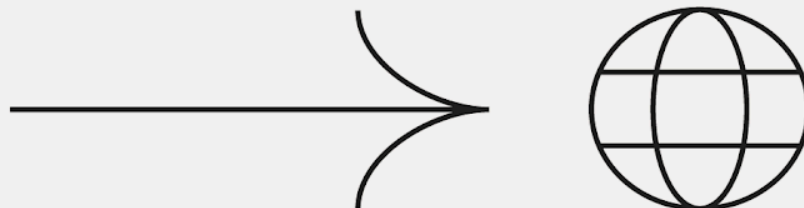


<https://www.baccan.it>

"Smetti di chattare, inizia a governare."



Google
Developer
Groups



2026



Google
Developer
Groups